

Applying Machine Learning Models to Software Development: A Quantitative Study of Multi-Agent LLM Configurations, Self-Healing Phenomena, and Agentic Paralysis in Software Engineering Benchmarks

Shahab Ali

Faculty of Artificial Intelligence Technologies (FATII)
ITMO University
Saint Petersburg, Russia
Group J4132, Student ID: 503271

Sergey Kovalchuk

Faculty of Artificial Intelligence Technologies (FATII)
ITMO University
Saint Petersburg, Russia

Abstract—This paper presents the empirical component of the research project *Applying Machine Learning Models to Software Development: Agent-Based Models*, conducted at ITMO University under the supervision of Professor Sergey Kovalchuk. Large Language Model (LLM)-based multi-agent systems are emerging as a promising approach to automating complex software engineering tasks. However, the performance impact of different role configurations and model scales on agentic framework stability remains poorly understood. Building upon a prior Systematic Literature Review of 19 state-of-the-art papers that produced a 7-layer agent role taxonomy, this paper presents a systematic empirical evaluation of four LLM/role configurations within the OpenHands CodeAct framework, tested across two standard benchmarks: HumanEval (15 tasks) and SWE-bench (6 tasks), totalling 21 experimental runs. We investigate two independent variables: LLM scale (local 7B vs. cloud 120B parameter models) and agent role configuration (single-agent Coder vs. multi-agent Coder+Tester pair programming). Our experiments reveal five original findings. First, local 7B models fail 100% of tasks due to XML schema brittleness, while 120B models achieve a 100% pass rate on HumanEval. Second, 120B models exhibit an emergent *Self-Healing* capability, autonomously recovering from framework errors in 90% of cases. Third, complex repository tasks induce *Agentic Paralysis*, a novel failure mode where agents correctly identify next actions but fail to execute them. Fourth, a single meta-cognitive prompt instruction raises SWE-bench pass rate from 33.3% to 100%. Fifth, standard CI/CD terminal noise causes context window exhaustion, requiring targeted mitigations. We document a taxonomy of 10 failure types and discuss implications for designing reliable agentic software engineering systems.

Index Terms—multi-agent systems, large language models, software engineering, OpenHands, HumanEval, SWE-bench, agentic paralysis, self-healing, prompt engineering

I. INTRODUCTION

This paper presents the empirical Semester 2 contribution of the research project *Applying Machine Learning Models to Software Development: Agent-Based Models* at ITMO University. The theoretical foundation, a 7-layer agent role taxonomy derived from a PRISMA-based Systematic Literature Review (SLR) of 19 state-of-the-art papers, was established in Semester 1. The present work transitions from theory to practice by deploying and benchmarking the taxonomy within the OpenHands CodeAct framework.

Software engineering is undergoing a fundamental shift from single-agent tools such as GitHub Copilot, which assist individual developers with code completion, towards Autonomous Multi-Agent Systems (MAS) in which specialised agents collaborate to solve complex software tasks [1], [9]. This transition was catalysed by the release of GPT-4 in 2023 and has accelerated rapidly, with frameworks such as MetaGPT [1], ChatDev [2], and Magentic-One [3] demonstrating multi-agent pipelines across the full software development lifecycle.

The core limitation of single agents is their inability to self-correct effectively. When a single agent encounters an error, it often hallucinates an incorrect response or fails silently. Multi-agent systems address this by separating responsibilities: a *Manager* plans, a *Coder* implements, and a *Tester* verifies and corrects the output autonomously.

Despite rapid progress, fundamental questions remain open. Specifically: (i) does model scale or role configuration more significantly determine agentic framework success? (ii) what failure modes emerge in agentic pipelines on standard benchmarks? and (iii) can targeted prompt engineering mitigate these failures?

This paper addresses these questions through a systematic empirical study. We make the following contributions:

- C1.** We present a controlled quantitative evaluation of four LLM/role configurations on HumanEval and SWE-bench benchmarks within the OpenHands CodeAct framework.
- C2.** We identify and name the *Self-Healing Phenomenon*: an emergent error-recovery capability present in 120B models but absent in 7B models.
- C3.** We identify and name *Agentic Paralysis*: a novel failure mode in which agents correctly reason about the next action but fail to execute the corresponding tool call.
- C4.** We demonstrate that a single meta-cognitive Anti-Paralysis instruction raises SWE-bench pass rate from 33.3% to 100%.
- C5.** We contribute a taxonomy of 10 failure types observed across 21 experimental runs.

II. BACKGROUND AND RELATED WORK

A. Multi-Agent LLM Frameworks

MetaGPT [1] encodes software team workflows as Standard Operating Procedures (SOPs), assigning roles such as Product Manager, Architect, and QA Engineer to separate LLM agents. ChatDev [2] implements a waterfall model with a Chat Chain structure enabling CEO, CTO, Programmer, and Reviewer agents. Magentic-One [3] introduces a Manager-Worker orchestration pattern with pluggable specialist agents. PotPie [4] grounds agent actions in a repository knowledge graph, enabling Debugger, Tester, and Designer roles with code-aware context. MapCoder [5] demonstrates a plan-implement-debug pipeline for competitive programming.

B. Agent Taxonomy

A prior Systematic Literature Review (SLR) conducted by the authors following PRISMA guidelines analysed 19 state-of-the-art papers from 2023–2025 and produced a 7-layer agent role taxonomy spanning Orchestration, Analysis & Planning, Design, Implementation, Quality Assurance (QA), Knowledge & Documentation, and Operations layers. A key finding was that the QA layer dominates the literature, with 9 of 19 papers focusing on verification and self-correction, confirming a shift from code generation towards code verification in the field. The present paper empirically validates this taxonomy.

C. The OpenHands CodeAct Framework

OpenHands [6] is an open-source platform for autonomous AI software agents, operating within an isolated Docker container with access to a bash terminal. The CodeAct architecture within OpenHands expresses agent actions as executable code rather than natural language, creating a Think→Act→Observe loop grounded in real code execution.

III. RESEARCH DESIGN

A. Research Questions

We investigate three research questions:

- **RQ1:** Does model scale (7B vs. 120B) or role configuration (single vs. multi-agent) more significantly determine pass rate on standard SE benchmarks?
- **RQ2:** What failure modes emerge in agentic pipelines and how do they relate to model scale?
- **RQ3:** Can meta-cognitive prompt engineering mitigate identified failure modes?

B. Agent Architecture

We implement three roles derived from the 7-layer taxonomy described in Section II-B:

Manager (Orchestration layer): receives the user prompt and decomposes it into a structured logic plan stored in `TASKS.md`. System prompt: “*You are the Project Manager. Your job is to take the user’s request and break it down into a step-by-step logic plan. Do not write code. Output only the logic steps required to solve the problem.*”

Coder (Implementation layer): receives the plan, writes an executable Python script, and saves it to the workspace. System prompt: “*You are the Coder. Take the step-by-step plan provided by the Manager and write the executable Python script. Save the file locally. Do not run the file.*”

Tester (Quality Assurance layer): executes the script, reads any error traces, and returns bugs to the Coder for correction. System prompt: “*You are the QA Tester. Execute the Python file in the terminal. If you encounter an error, read the terminal trace and send it back to the Coder for a fix. If it runs cleanly, output PASS.*”

The resulting workflow is:

Human → Manager → Coder ↔ Tester → Output

where the bidirectional Coder–Tester loop operates autonomously until all tests pass.

C. Experimental Configurations

We isolate two independent variables to form four configurations (Table I):

TABLE I
LLM/ROLE CONFIGURATIONS EVALUATED

Config	LLM	Role	Setup
1	Local 7B	Single	Coder only
2	Local 7B	Multi	Coder + Tester
3	Cloud 120B	Single	Coder only
4	Cloud 120B	Multi	Coder + Tester

D. Experimental Infrastructure

The evaluation uses OpenHands CodeAct deployed via Docker on Ubuntu/WSL2. The local 7B model was served via Ollama (Llama 3.1 / CodeLlama). The cloud 120B model (gpt-oss-120b) was accessed via the Cerebras API following OpenRouter server downtime. Benchmarks: HumanEval [7] (15 function-level Python tasks) and SWE-bench [8] (6 repository-level tasks). Total experimental runs: 21.

E. Metrics

For each run we record: *Pass Rate* (task solved correctly), *Crash Rate* (fatal XML schema failure causing total framework collapse), *Self-Healing Rate* (agent autonomously recovered from a framework error without human intervention), and *Paralysis Events* (reasoning loops where agent narrates intended actions without executing them).

IV. RESULTS

A. Phase 1: HumanEval Benchmark

Table II presents aggregated HumanEval results by configuration across 15 tasks.

TABLE II
HUMANEval RESULTS BY CONFIGURATION

Cfg	Role	Tasks	Pass%	Crash%	Heal%
1	Single 7B	5	0%	100%	0%
2	Multi 7B	5	0%	100%	0%
3	Single 120B	5	100%	0%	100%
4	Multi 120B	10	100%	0%	100%

Local 7B models crashed on 100% of tasks under both role configurations. Failure analysis revealed the root cause was not incorrect Python logic but rather the inability to adhere to the strict XML tool-calling schema enforced by OpenHands v0.59, which requires a mandatory `security_risk` parameter in all `str_replace_editor` and `execute_bash` calls. The 7B model hallucinated malformed tags (`<parameter=command>`) rather than the required JSON-structured parameter block, causing the framework to reject all tool calls and crash.

Cloud 120B models passed 100% of tasks. Notably, the 120B model encountered the same `security_risk` schema error in 8 of 15 tasks but autonomously recovered in all 8 cases by reading its own error trace, identifying the missing parameter, and re-issuing the corrected tool call. Additional self-healing events included file conflict resolution (Task #9: agent pivoted from `create` to `view` upon detecting existing file) and placeholder hallucination recovery (Task #14: agent wrote literal `...code...` text, diagnosed the resulting `SyntaxError`, and rewrote the correct function).

Configuration 4 (120B Multi-Agent) produced the most comprehensive test suites, averaging 5–8 unit test cases per task compared to 0 in Configuration 3, confirming the value of the Tester role even when pass rates are equivalent.

B. Phase 2: SWE-bench Benchmark

Table III presents SWE-bench results. All six tasks used Configuration 4 (120B Multi-Agent).

SWE-bench #2 failed due to a missing Flask dependency in the execution environment. The agent correctly diagnosed the root cause (`ModuleNotFoundError`) but could not self-heal as the fix required system-level `pip install`, which was explicitly prohibited in the prompt constraints.

SWE-bench #3 produced the most severe failure in the study. After successfully reading the main implementation file, the

TABLE III
SWE-BENCH RESULTS (#1–#6)

Task	Files	Anti-P.	Result	Finding
#1	calc + app	No	PASS	Agentic Loop
#2	app.py	No	FAIL	Env. Dep.
#3	calculator.py	No	FAIL	Chain Fail
#4	math_utils	Yes	PASS	8/8 tests
#5	humaneval_res	Yes	PASS	11/11 tests
#6	calc extended	Yes	PASS	Recovered

agent entered a Verbose Agentic Paralysis loop, repeating the statement “We need to view test file” over 200 times without executing the corresponding tool call. The 200+ repetitions consumed sufficient tokens to exceed the Cerebras TPM rate limit, triggering a cascade Chain Failure: Paralysis → Token Exhaustion → API Crash. The agent exhibited partial self-awareness, stating “I’m stuck, I need to break out,” but the session had already failed.

Anti-Paralysis results (Table IV) show the impact of adding a single meta-cognitive constraint:

TABLE IV
ANTI-PARALYSIS PROMPT ENGINEERING IMPACT

Condition	Tasks	Pass Rate
Without Anti-Paralysis	3	33.3%
With Anti-Paralysis	3	100%

The Anti-Paralysis instruction added was: “*If any single action fails or repeats more than 3 times, STOP immediately. Run: `ls /workspace/` to reorient. Maximum 3 attempts per action.*”

V. KEY FINDINGS

A. Finding 1: Framework Brittleness vs. Model Scale

Model scale, not role configuration, is the primary determinant of agentic framework success. 7B models crash 100% of tasks regardless of assigned role due to XML schema brittleness. 120B models pass 100% of HumanEval tasks regardless of role. Role configuration influences output quality (test suite depth, code elegance) but not binary pass/fail outcomes for the 120B model. This finding implies that practitioners must prioritise model capacity before optimising role architecture.

B. Finding 2: The Self-Healing Phenomenon

The 120B model demonstrates an emergent Self-Healing capability, autonomously recovering from framework errors in 9 of 10 error events (90% recovery rate). This capability is completely absent in 7B models. Crucially, Task #14 revealed that Self-Healing extends beyond API schema errors to recovery from the model’s own incorrect output (placeholder hallucination), suggesting this is a *general* error-recovery capability rather than a narrow one. We hypothesise that this emerges from the model’s ability to maintain sufficient context to compare its intended output against the observed error trace—a capability that degrades below a critical parameter threshold.

C. Finding 3: Agentic Paralysis

We define *Agentic Paralysis* as a failure mode in which an agent correctly identifies the required next action through reasoning but fails to execute the corresponding tool call. Two variants were observed: *Mild Paralysis* (5–6 repetitions, self-resolving, SWE #6) and *Verbose Paralysis* (200+ repetitions, leading to cascade failure, SWE #3). The cascade pattern Paralysis → Token Exhaustion → API Crash represents a previously undocumented failure mode in LLM-based agentic systems.

D. Finding 4: Anti-Paralysis Prompt Engineering

A single meta-cognitive constraint raised SWE-bench pass rate from 33.3% to 100%, demonstrating that targeted prompt engineering can effectively prevent Agentic Paralysis without modifying the underlying model or framework. This finding has immediate practical implications: the instruction is model-agnostic, framework-agnostic, and adds negligible overhead.

E. Finding 5: Context Window Exhaustion

Standard CI/CD terminal output (verbose `pip install` progress bars, full `pytest` stack traces) floods the agent context window, causing TPM rate-limit crashes. Mitigations developed: `-q` flag on `pytest`, `head -n 20` output truncation, pre-installation of dependencies, and targeted test file selection to prevent cross-contamination.

VI. FAILURE TAXONOMY

Table V presents the complete taxonomy of 10 failure types identified across 21 experimental runs.

TABLE V
FAILURE TAXONOMY (10 TYPES, 21 RUNS)

#	Type	Recovery	Prevention
1	XML Schema Error	Self-Heal	Partial
2	Context Collapse	None	Clear prompts
3	Prompt Bleed	None	Simpler prompts
4	File Conflict	Pivot	N/A
5	Placeholder Halluc.	Self-Heal	Partial
6	Agentic Paralysis	None	Anti-Paralysis
7	Verbose Paralysis	Partial	Anti-Paralysis
8	Chain Failure	None	Anti-Paralysis
9	Env. Dependency	None	Pre-install
10	File Not Found	Explore	Check workspace

Failures 1, 4, and 5 were exclusively self-healed by 120B models. Failures 2, 3, and 9 are irrecoverable without external intervention. Failures 6, 7, and 8 are preventable through Anti-Paralysis prompt engineering. The 7B model exhibited Failures 2 and 3 exclusively without any recovery.

VII. DISCUSSION

A. Taxonomy Validation

The Semester 1 SLR predicted that the QA layer would dominate multi-agent software engineering, with 9 of 19 surveyed papers focusing on verification and self-correction. This prediction is empirically confirmed: the Self-Healing

Phenomenon is fundamentally a QA behaviour in which the agent acts as its own tester, verifying its output against observed execution results and issuing corrective actions autonomously. Single agents that lack the Tester role cannot exhibit this behaviour, which explains why single-agent tools struggle with complex, error-prone tasks.

B. Implications for System Design

Our results yield three practical recommendations for practitioners building LLM-based software engineering agents:

R1: Minimum model scale threshold. Agentic frameworks require a minimum model capacity to function. Based on our results, models below 10B parameters are unreliable for tool-calling schemas in modern sandboxes. Practitioners should test framework compatibility before investing in role engineering.

R2: Anti-Paralysis as a default. The Anti-Paralysis instruction should be included as a default system prompt component in all agentic deployments, given its zero-cost, model-agnostic nature and significant impact (33.3% to 100% SWE-bench improvement).

R3: Terminal output management. Agents deployed in CI/CD environments must truncate terminal output by default (`-q` flag, `head -n 20`) to prevent context window exhaustion during log-heavy operations.

C. Limitations

This study has three main limitations. First, the evaluation is limited to 21 runs across two benchmarks; larger-scale evaluation is needed to generalise the Self-Healing and Agentic Paralysis findings. Second, only one cloud model (gpt-oss-120b at 120B parameters) was evaluated; it is unclear whether the observed phenomena generalise to other model families at similar scales. Third, a full three-role Manager+Coder+Tester configuration was not formally benchmarked on HumanEval; this is left for future work.

VIII. CONCLUSION

This paper presents a systematic empirical evaluation of multi-agent LLM configurations for software engineering tasks within the OpenHands CodeAct framework. Across 21 experimental runs on HumanEval and SWE-bench, we find that model scale is the primary determinant of framework success, that 120B models exhibit an emergent Self-Healing capability that 7B models lack entirely, and that a single Anti-Paralysis prompt instruction eliminates Agentic Paralysis and raises SWE-bench pass rate from 33.3% to 100%.

These findings connect to a broader theoretical prediction from our prior systematic literature review: the shift from code generation to code verification is the defining trend in multi-agent software engineering. The Self-Healing Phenomenon confirms this empirically: the most impactful capability of large-scale agents is not writing correct code on the first attempt, but correctly diagnosing and fixing their own mistakes.

Future work will extend the evaluation to a full three-role Manager+Coder+Tester configuration, test additional model families, and investigate whether the Agentic Paralysis failure

mode can be detected and prevented at the framework level rather than through prompt engineering.

ACKNOWLEDGEMENTS

The author thanks Professor Sergey Kovalchuk (ITMO University) for supervision and guidance throughout this research project.

REFERENCES

- [1] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” *arXiv:2308.00352*, 2023.
- [2] C. Qian et al., “ChatDev: Communicative Agents for Software Development,” in *Proc. ACL 2024*, Bangkok, Thailand, 2024.
- [3] A. Fournay et al., “Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks,” *Microsoft Research*, 2024.
- [4] PotPie AI, “PotPie: Custom Engineering Agents Grounded in Your Codebase,” <https://github.com/potpie-ai/potpie>, 2025.
- [5] Md.A. Islam et al., “MapCoder: Multi-Agent Code Generation for Competitive Problem Solving,” in *Proc. ACL 2024*.
- [6] X. Wang et al., “OpenHands: An Open Platform for AI Software Developers as Generalist Agents,” *arXiv:2407.16741*, 2024.
- [7] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv:2107.03374*, 2021.
- [8] C.E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in *Proc. ICLR 2024*.
- [9] Z. Rasheed et al., “LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead,” *arXiv:2404.04834*, 2024.
- [10] C.S. Xia et al., “Agentless: Demystifying LLM-based Software Engineering Agents,” *arXiv:2407.01489*, 2024.
- [11] T. Händler, “A Taxonomy for Autonomous LLM-Powered Multi-Agent Architectures,” *arXiv:2310.03659*, 2024.
- [12] X. Tang et al., “CodeAgent: Autonomous Communicative Agents for Code Review,” *arXiv:2402.02172*, 2024.
- [13] J. Wang et al., “Autonomous Agents for Software Testing,” *arXiv:2409.09030*, 2024.
- [14] J. Liu et al., “From LLMs to LLM-based Agents for Software Engineering: A Survey,” *arXiv:2408.02479*, 2024.
- [15] S. Kapoor et al., “AI Agents: Evolution, Architecture, and Real-World Applications,” *arXiv:2503.12687*, 2025.